

ООП 2

ШАБЛОНИ ЗА ДИЗАЙН. ШАБЛОНИ ЗА СЪЗДАВАНЕ

СЪЩНОСТ

- КАКВО Е ШАБЛОН
- КАКВО Е СОФТУЕРЕН ШАБЛОН
 - ДОКАЗАНИ ТЕХНИКИ ЗА РЕШАВАНЕ НА ДАДЕН СОФТУЕРЕН ПРОБЛЕМ

ЕЛЕМЕНТИ НА ШАБЛОНА

- ИМЕ НА ШАБЛОНА – ОПИСАНИЕ НА ПРОБЛЕМА
- ПРОБЛЕМ ЗА РЕШАВАНЕ
- РЕШЕНИЕ – ОПИСВА ЕЛЕМЕНТИТЕ И ТЯХНОТО ВЗАИМОДЕЙСТВИЕ В РЕШЕНИЕТО НА ПРОБЛЕМА
- ПОСЛЕДСТВИЯ

НЕОБХОДИМОСТ

- ГЪВКАВОСТ
 - ОСИГУРЯВА ПРАВИЛНОТО НИВО НА АБСТРАКЦИЯ
 - ОБЕКТите СТАВАТ СЛАБО СВЪРЗАНИ ЕДИН С ДРУГ – КОД ЛЕСЕН ЗА ПРОМЯНА
 - ЛЕСЕН ЗА ТЕСТВАНЕ
- ПРЕИЗПОЛЗВАНЕ НА КОД
- ИЗПОЛЗВАНЕ НА ДОБРИ ПРАКТИКИ УСПЕШНИ ДИЗАЙНИ И АРХИТЕКТУРИ (ДОКАЗАНИ РЕШЕНИЯ)
- ПОДОБРЯВАТ ДОКУМЕНТАЦИЯТА
 - ЕКЦПЛИЦИТНИ ВЗАИМОДЕЙСТВИЯ

ИЗБОР И ИЗПОЛЗВАНЕ

- МНОГО НА БРОЙ
- БЛИЗКИ ПО ПРЕДНАЗНАЧЕНИЕ И ИМПЛЕМЕНТАЦИЯ
- ОПРЕДЕЛЯНЕ СПОРЕД ПРОБЛЕМА ЗА РЕШАВАНЕ
 - ШАБЛОНИ ЗА СЪЗДАВАНЕ
 - СТРУКТУРНИ ШАБЛОНИ
 - ШАБЛОНИ ЗА ПОВЕДЕНИЕ

ИЗБОР И ИЗПОЛЗВАНЕ

- НАЛИЧИЕ НА МНОГО ИНСТАНЦИИ КОИТО ПРЕДСТАВЛЯВАТ ЕДНО И СЪЩО НЕЩО
 - ШАБЛОН ЕДИНИЧНА ИНСТАНЦИЯ (SINGLETON)
- СИЛНА ЗАВИСИМОСТ МЕЖДУ КЛАСОВЕ
 - ШАБЛОНИ „МОСТ“ (BRIDGE), „ПОСРЕДНИК“ (MEDIATOR) ИЛИ „КОМАНДА“ (COMMAND)
- НАЛИЧИЕ НА ДВА РАЗЛИЧНИ НЕСЪВМЕСТИМИ ИНТЕРФЕЙСА (ОБЕКТА)
 - АДАПТЕР ШАБЛОН
- ПРОБЛЕМ – ШАБЛОН M:N

ТИПОВЕ ШАБЛОНИ

- ШАБЛОНИ ЗА СЪЗДАВАНЕ
 - ДЕФИНИЦИЯ ЗА СЪЗДАВАНЕ НА ОБЕКТ
- СТРУКТУРНИ ШАБЛОНИ
 - ДЕФИНИЦИЯ КАК ДА СЕ КОМПОЗИРАТ КЛАСОВЕ ЗА ДА СЕ ПОЛУЧИ ПО-ГОЛЯМА СТРУКТУРА
- ШАБЛОНИ ЗА ПОВЕДЕНИЕ
 - ДЕФИНИЦИЯ ЗА ВЗАИМОДЕЙСТВИЕ МЕЖДУ ОБЕКТИ

СТРУКТУРНИ ШАБЛОНИ. ЕДИНИЧНА ИНСТАНЦИЯ (SINGLETON)

- НЕОБХОДИМОСТ ОТ САМО ЕДНА ИНСТАНЦИЯ НА ДАДЕН ОБЕКТ
 - КОНТРОЛ НА ИНСТАНЦИЯТА - ДА ИМА ТОЧНО ЕДНА ИНСТАНЦИЯ НА КЛАС И ТОЙ ТРЯБВА ДА Е ДОСТЪПЕН ОТ ДОБРЕ ПОЗНАТА ТОЧКА ЗА ДОСТЪП (КОНТРОЛИРАН ДОСТЪП)

ИМПЛЕМЕНТАЦИЯ НА SINGLETON

- EAGER INITIALIZATION
- STATIC BLOCK INITIALIZATION
- LAZY INITIALIZATION
- THREAD SAFE INITIALIZATION
- BILL PUGH INITIALIZATION

ИМПЛЕМЕНТАЦИЯ НА SINGLETON

- ```
PUBLIC CLASS SINGLETONEAGER {
 PUBLIC STATIC SINGLETONEAGER SC = NEW SINGLETONEAGER();
}
```



# ИМПЛЕМЕНТАЦИЯ НА SINGLETON

- ```
PUBLIC CLASS SINGLETONEAGER {  
    PUBLIC STATIC SINGLETONEAGER SC = NEW SINGLETONEAGER();  
    PRIVATE SINGLETONEAGER(){}  
}
```

ИМПЛЕМЕНТАЦИЯ НА SINGLETON

- ```
PUBLIC CLASS SINGLETONEAGER {
 PRIVATE STATIC SINGLETONEAGER SC = NEW SINGLETONEAGER();
 PRIVATE SINGLETONEAGER(){}
 PUBLIC STATIC SINGLETONEAGER GETINSTANCE(){
 RETURN SC;
 }
}
```



# ИМПЛЕМЕНТАЦИЯ НА SINGLETON

- ```
PUBLIC CLASS SINGLETONLAZY {  
    PRIVATE STATIC SINGLETONLAZY SC = NULL;  
  
    PRIVATE SINGLETONLAZY(){}  
  
    PUBLIC STATIC SINGLETONLAZY GETINSTANCE(){  
        IF(SC==NULL){  
            SC = NEW SINGLETONLAZY();  
        }  
  
        RETURN SC;  
    }  
}
```

- МНОЖЕСТВЕН ДОСТЪП?

Singleton
- <u>singleton : Singleton</u>
- Singleton()
+ <u>getInstance() : Singleton</u>

ИМПЛЕМЕНТАЦИЯ НА SINGLETON

```
PUBLIC CLASS BILLPUGH SINGLETON {  
    PRIVATE STATIC CLASS SINGLETONHELPER {  
        PRIVATE STATIC FINAL BILLPUGH SINGLETON INSTANCE = NEW BILLPUGH SINGLETON();  
    }  
    PRIVATE BILLPUGH SINGLETON() { }  
    PUBLIC STATIC BILLPUGH SINGLETON GETINSTANCE() {  
        RETURN SINGLETONHELPER.INSTANCE;  
    }  
}
```


НАЧИНИ ЗА НАРУШАВАНЕ НА ЕДИНИЧНИЯ МОДЕЛ

- КЛОНИРАНЕ.
- REFLECTION.
- НАЛИЧИЕ НА НЯКОЛКО КЛАС LOADERS
- СЕРИАЛИЗАЦИЯ

ШАБЛОН СТРОИТЕЛ (BUILDER PATTERN)

- КАК СЕ СЪЗДАВА ОБЕКТ?
 - ИЗПОЛЗВАНЕ НА КОНСТРУКТОР
 - КОГА Е ВЪЗМОЖНО? (КОГА Е ЕФЕКТИВНО)
 - ОБЕКТЪТ, КОЙТО СЕ СЪЗДАВА, Е СЛОЖЕН - ?
 - СЪЗДАВАНЕ ПО РАЗЛИЧНИ НАЧИНИ — РАЗЛИЧНИ ПРЕЗЕНТАЦИИ (РАЗЛИЧНИ ПАРАМЕТРИ)
 - РАЗЛИЧНИ ЛОГИКИ ЗА СЪЗДАВАНЕ

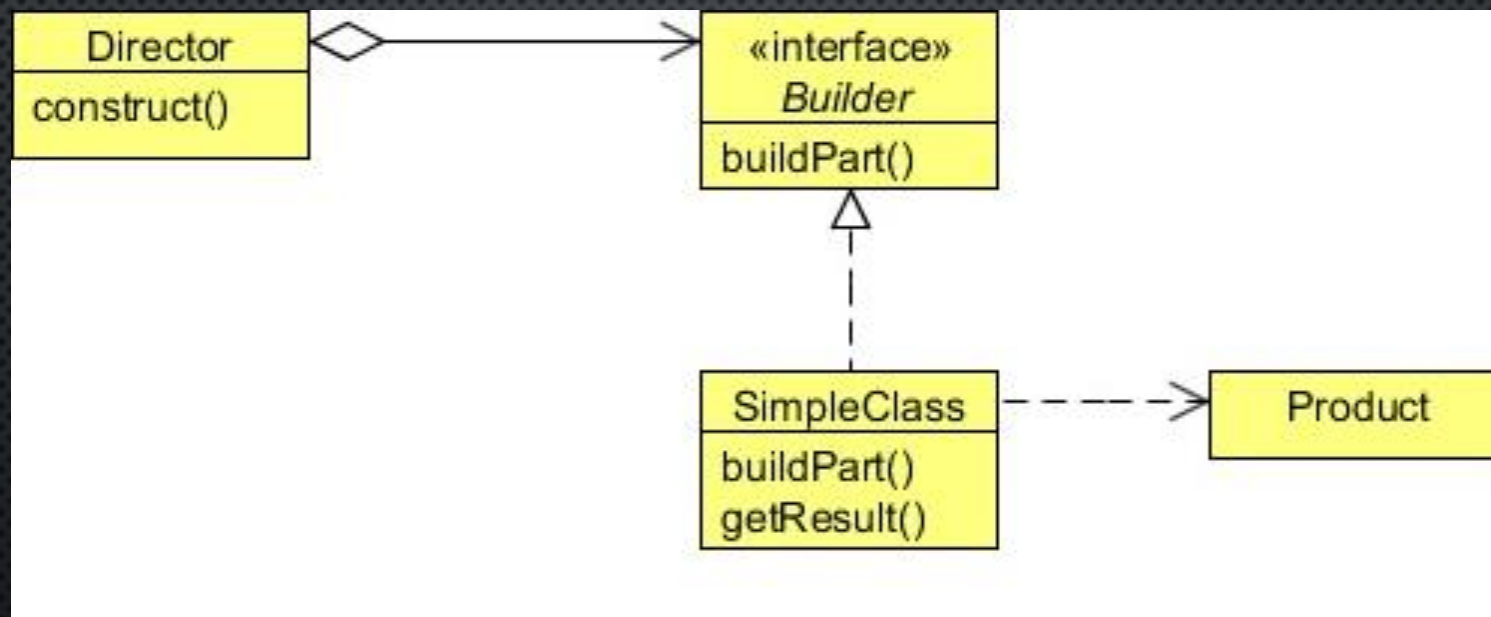
ШАБЛОН СТРОИТЕЛ (BUILDER PATTERN)

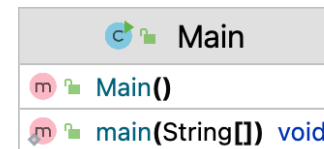
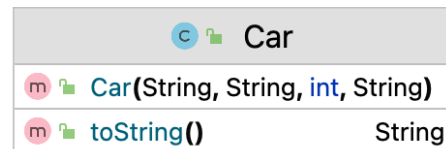
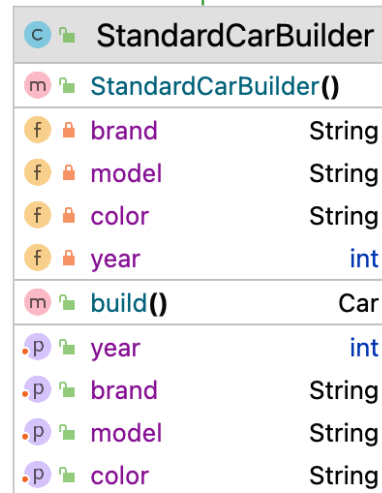
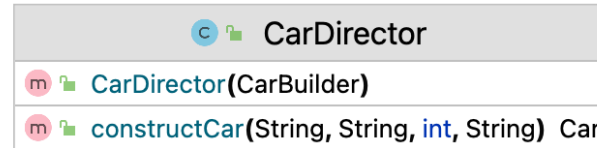
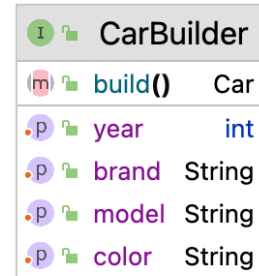
- ТЕЛЕСКОПИЧНИ КОНСТРУКТОРИ
- ШАБЛОН СТРОИТЕЛ

ШАБЛОН СТРОИТЕЛ (BUILDER PATTERN)

- КОНЦЕПЦИЯ – ОТДЕЛЯНЕ НА СЪЗДАВАНЕТО НА ОБЕКТА ОТ НЕГОВАТА ПРЕЗЕНТАЦИЯ
- МНОЖЕСТВО НАЧИНИ ЗА СЪЗДАВАНЕ

ШАБЛОН СТРОИТЕЛЯ (BUILDER PATTERN)





- ```
PUBLIC CLASS CAR {
 PRIVATE STRING BRAND;
 PRIVATE STRING MODEL;
 PRIVATE INT YEAR;
 PRIVATE STRING COLOR;

 PUBLIC CAR(STRING BRAND, STRING MODEL, INT YEAR, STRING COLOR) {
 THIS.BRAND = BRAND;
 THIS.MODEL = MODEL;
 THIS.YEAR = YEAR;
 THIS.COLOR = COLOR;
 }

 @OVERRIDE
 PUBLIC STRING toString() {
 RETURN "CAR{" +
 "BRAND='" + BRAND + "\" +
 ", MODEL='" + MODEL + "\" +
 ", YEAR=" + YEAR +
 ", COLOR='" + COLOR + "\" +
 '}';
 }
}
```

# ШАБЛОН СТРОИТЕЛЯ (BUILDER PATTERN) МОДЕЛЬ

- `PUBLIC INTERFACE CARBUILDER {`  
    `CARBUILDER SETBRAND(String BRAND);`  
    `CARBUILDER SETMODEL(String MODEL);`  
    `CARBUILDER SETYEAR(int YEAR);`  
    `CARBUILDER SETCOLOR(String COLOR);`  
    `CAR BUILD();`  
}





# ШАБЛОН СТРОИТЕЛЯ (BUILDER PATTERN) МОДЕЛЬ

- ```
PUBLIC CLASS CARDIRECTOR {  
    PRIVATE CARBUILDER BUILDER;  
  
    PUBLIC CARDIRECTOR(CARBUILDER BUILDER) {  
        THIS.BUILDER = BUILDER;  
    }  
  
    PUBLIC CAR CONSTRUCTCAR(String BRAND, String MODEL, INT YEAR, String COLOR) {  
        RETURN BUILDER.SETBRAND(BRAND)  
            .SETMODEL(MODEL)  
            .SETYEAR(YEAR)  
            .SETCOLOR(COLOR)  
            .BUILD();  
    }  
}
```


ШАБЛОН СТРОИТЕЛ (BUILDER PATTERN) МОДЕЛ

```
•  
PUBLIC CLASS MAIN {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        // Създаване на директор  
        CARBUILDER BUILDER = NEW STANDARDCARBUILDER();  
        CARDIRECTOR DIRECTOR = NEW CARDIRECTOR(BUILDER);  
  
        // Създаване на стандартен автомобил чрез директора  
        CAR CAR = DIRECTOR.CONSTRUCTCAR("TOYOTA", "COROLLA", 2022, "RED");  
  
        // Извеждане на създадения автомобил  
        SYSTEM.OUT.PRINTLN(CAR);  
    }  
}
```

Car{brand='Toyota', model='Corolla', year=2022, color='Red'}

ШАБЛОН СТРОИТЕЛ НУЛИРАНЕ НА СЪСТОЯНИЕ

- `RESET()` МЕТОД НА СТРОИТЕЛЯ

КОГА ДА ИЗПОЛЗВАМЕ

- АЛГОРИТЪМЪТ ЗА СЪЗДАВАНЕ НА СЛОЖЕН ОБЕКТ ТРЯБВА ДА БЪДЕ НЕЗАВИСИМ ОТ ЧАСТИТЕ, КОИТО СЪСТАВЛЯВАТ ОБЕКТА И КАК ТЕ СА СГЛОБЕНИ.
- ПРОЦЕСЪТ НА ИЗГРАЖДАНЕ ТРЯБВА ДА ПОЗВОЛЯВА РАЗЛИЧНИ ПРЕЗЕНТАЦИИ НА КОНСТРУИРАНИЯ ОБЕКТ